

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

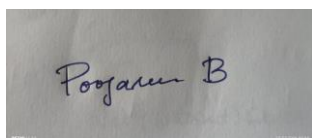
Weightage: 40%

QUESTIONS:

Total Marks: 30

Code of Conduct:

I POOJASREE B (192411091) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

A photograph of a piece of paper with a handwritten signature in blue ink. The signature appears to be 'Poojasree B'.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 1: Servlet-Based Student Registration System

1. Architecture of a Servlet-Based Application

A servlet-based web application follows a **client–server architecture** in which the browser sends an HTTP request to a web server, and a Java Servlet processes the request and generates a response.

Main components

1. Client (Web Browser)

- a. The student enters registration details in an HTML form.
- b. The browser sends the form data to the server using either the GET or POST HTTP method.

2. Web Server / Servlet Container

- a. Examples include Apache Tomcat and Jetty.
- b. The servlet container receives the HTTP request.
- c. It identifies the appropriate servlet based on the URL mapping.
- d. It manages the servlet's lifecycle and provides request/response objects.

3. Servlet

- a. The servlet contains Java code that processes the submitted student information.
- b. It reads form parameters using methods such as `request.getParameter()`.
- c. It can validate the data, perform business logic, and interact with a database.

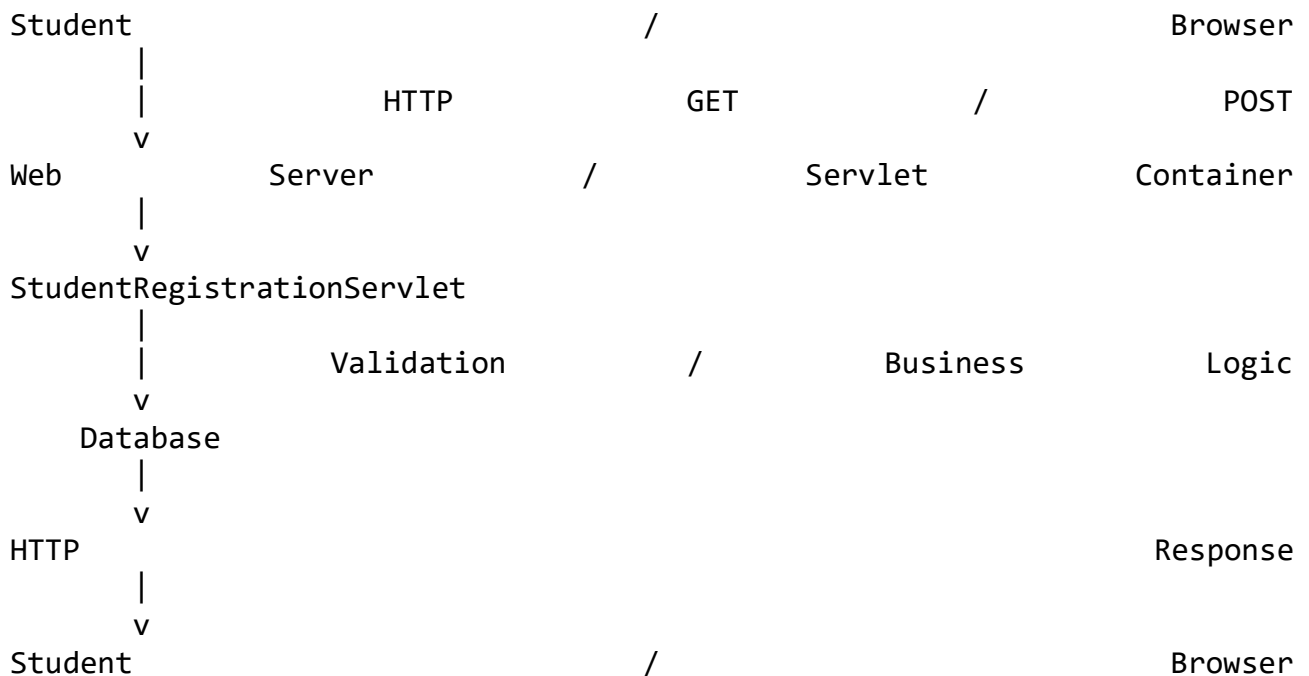
4. Database

- a. Student details can be stored in a database such as MySQL or PostgreSQL.
- b. The servlet communicates with the database using JDBC or another persistence framework.

5. HTTP Response

- a. After processing the request, the servlet sends an HTTP response back to the browser.
- b. The response may contain HTML, a confirmation message, or a redirect to another page.

Basic architecture



2. HTML Form for Student Registration

The following form collects basic student information.

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Registration</title>
</head>
<body>

<h2>Student Registration Form</h2>

<form action="register" method="post">
    Name:
    <input type="text" name="name" required><br><br>

    Email:
    <input type="email" name="email" required><br><br>

    Course:
    <input type="text" name="course" required><br><br>

    Password:
    <input type="password" name="password" required><br><br>

    <input type="submit" value="Register">
</form>
```

```
</body>
</html>
```

The form sends the data to the servlet mapped to /register.

3. Servlet Implementation Using GET and POST

Using the modern Jakarta Servlet API:

```
import java.io.IOException;
import java.io.PrintWriter;

import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/register")
public class StudentRegistrationServlet extends HttpServlet {

    @Override
    public void init() throws ServletException {
        System.out.println("Servlet initialized.");
    }

    // Handles GET requests
    @Override
    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");

        PrintWriter out = response.getWriter();

        String name = request.getParameter("name");
        String email = request.getParameter("email");
        String course = request.getParameter("course");

        out.println("<html><body>");
        out.println("<h2>Student Details Received Using GET</h2>");
        out.println("<p>Name: " + name + "</p>");

        out.println("<p>Email: " + email + "</p>");
        out.println("<p>Course: " + course + "</p>");
        out.println("</body></html>");
    }

    // Handles POST requests
    @Override
```

```

protected void doPost(HttpServletRequest request,
                        HttpServletResponse response)
    throws ServletException, IOException {

    response.setContentType("text/html");

    String name = request.getParameter("name");
    String email = request.getParameter("email");
    String course = request.getParameter("course");
    String password = request.getParameter("password");

    // Example validation
    if (name == null || email == null ||
        course == null || password == null ||
        name.isBlank() || email.isBlank() ||
        course.isBlank() || password.isBlank()) {

        response.getWriter().println(
            "<h3>All fields are required.</h3>"
        );
        return;
    }

    PrintWriter out = response.getWriter();

    out.println("<html><body>");
    out.println("<h2>Registration Successful</h2>");
    out.println("<p>Welcome, " + name + "!</p>");
    out.println("<p>Email: " + email + "</p>");
    out.println("<p>Course: " + course + "</p>");
    out.println("<p>Your registration has been received.</p>");
    out.println("</body></html>");
}

@Override
public void destroy() {
    System.out.println("Servlet destroyed.");
}
}

```

Note: If the application uses the older Java EE API, `javax.servlet.*` can be used instead of `jakarta.servlet.*`, depending on the servlet container version.

4. How GET and POST Work

GET Method

With GET, form parameters are appended to the URL.

Example:

</register?name=Rahul&email=rahul@example.com&course=BCA>

The servlet receives these values through:

```
request.getParameter("name");
request.getParameter("email");
request.getParameter("course");
```

GET is suitable for retrieving or searching information where the request does not change server-side data.

POST Method

With POST, form data is sent in the **HTTP request body** rather than being appended to the URL.

The servlet processes the data in `doPost()`:

```
protected void doPost(HttpServletRequest request,
                        HttpServletResponse response)
```

POST is generally preferred when submitting registration information because the data is not exposed in the URL.

5. Servlet Life Cycle

The servlet container controls the servlet's life cycle. The major stages are:

1. `init()`

The `init()` method is called **once**, when the servlet is created and initialized.

```
public void init() throws ServletException {
    System.out.println("Servlet initialized.");
}
```

It can be used for one-time initialization tasks such as loading configuration or establishing required resources.

2. `service()`

The `service()` method is called whenever the servlet receives a client request.



The `HttpServlet` class internally dispatches the request to the appropriate method, such as `doGet()` or `doPost()`.

Thus, programmers normally override `doGet()` and `doPost()` rather than directly overriding `service()`.

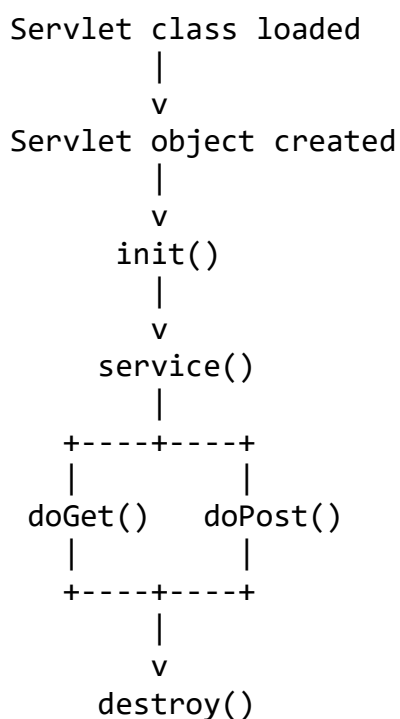
3. `destroy()`

The `destroy()` method is called **once** before the servlet is removed from service.

```
public void destroy() {  
    System.out.println("Servlet destroyed.");  
}
```

It is used for cleanup operations, such as releasing resources.

Life-cycle sequence



6. Which Method Is More Appropriate for Sensitive Data?

POST is more appropriate than GET for sensitive student registration data.

Reasons:

1. Data is not placed in the URL

- GET places parameters directly in the URL.
- URLs can appear in browser history, bookmarks, proxy logs, and server logs.

- c. POST places the form data in the HTTP request body.

2. Better suited for form submission

- a. POST is designed for operations that submit data to the server.
- b. Student registration normally creates or modifies server-side data.

3. Passwords and confidential information

- a. Sensitive values such as passwords should not be sent using GET because they would appear in the URL.
- b. POST avoids exposing them through the URL.

However, **POST alone does not encrypt data**. For sensitive information, the application should use **HTTPS/TLS**. Passwords should also never be stored as plain text; they should be securely hashed using an appropriate password-hashing algorithm.

7. Conclusion

A servlet-based student registration system consists of a client browser, web server/servlet container, servlet, business logic, and database. The servlet container manages the servlet's life cycle through `init()`, `service()`, and `destroy()`. `doGet()` is appropriate for retrieving information, while `doPost()` is preferable for submitting registration information and sensitive data. For real-world security, **POST should be combined with HTTPS and secure password handling**.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using `HttpSession` and `Cookies` in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Session Management Using `HttpSession` and `Cookies` in Servlets

1. Introduction

HTTP is a **stateless protocol**, which means that the web server does not automatically remember a user between different HTTP requests.

For example, after a user logs in, the server must be able to identify that user when they visit pages such as:

- Home page
- Profile page
- Dashboard
- Shopping cart
- Logout page

Session tracking is used to maintain this information across multiple requests.

In Java Servlets, session tracking can be implemented using **HttpSession** and **Cookies**.

2. Session Management Using HttpSession

HttpSession provides a convenient way to store information about a particular user on the server.

When a user logs in successfully, a session can be created and the user's information can be stored in it.

Login Servlet

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import jakarta.servlet.http.HttpSession;

@WebServlet("/login")

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                           HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        // Example authentication
        if ("admin".equals(username) && "1234".equals(password)) {
```

```

        HttpSession session = request.getSession();
        session.setAttribute("username", username);

        response.sendRedirect("home");
    } else {
        response.getWriter().println("Invalid username or
password");
    }
}
}

```

After successful authentication:

```

HttpSession session = request.getSession();
session.setAttribute("username", username);

```

The username is stored on the server in the user's session.

3. Accessing the Session on Other Pages

Once the user is logged in, other servlets can retrieve the session and check whether the user is authenticated.

```

import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/home")
public class HomeServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session != null &&
            session.getAttribute("username") != null) {

            String username =
                (String) session.getAttribute("username");

            response.getWriter().println(
                "<h2>Welcome, " + username + "!</h2>"
            );

        } else {
            response.sendRedirect("login.html");
        }
    }
}

```

The statement:

```
request.getSession(false);
```

returns the existing session if one exists. It does **not** create a new session.

This is useful when protecting pages that require authentication.

4. Logout and Session Invalidation

When the user logs out, the session should be destroyed.

```
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/logout")
public class LogoutServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session != null) {
            session.invalidate();
        }

        response.sendRedirect("login.html");
    }
}
```

The method:

```
session.invalidate();
```

terminates the session and removes the session data associated with the user.

5. Using Cookies for Session Tracking

A **cookie** is a small piece of information that a server sends to the client's browser. The browser stores it and sends it back to the server with subsequent requests.

Servlets provide the `Cookie` class for creating and reading cookies.

Creating a Cookie

```

Cookie    userCookie    =    new    Cookie("username",    username);

userCookie.setMaxAge(60 * 60);           //    1    hour
userCookie.setHttpOnly(true);

response.addCookie(userCookie);

```

Reading a Cookie

```

Cookie[]    cookies    =    request.getCookies();

String    username    =    null;

if    (cookies    !=    null)    {
    for    (Cookie    cookie    :    cookies)    {
        if    ("username".equals(cookie.getName()))    {
            username    =    cookie.getValue();
            break;
        }
    }
}

```

The servlet can then use the cookie value to identify the user.

6. Cookie-Based Login Example

```

@WebServlet("/cookieLogin")
public class CookieLoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                           HttpServletResponse response)
        throws IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        if ("admin".equals(username) && "1234".equals(password)) {

            Cookie cookie =
                new Cookie("username", username);

            cookie.setMaxAge(60 * 60);
            cookie.setHttpOnly(true);

            response.addCookie(cookie);

            response.sendRedirect("cookieHome");

        } else {
            response.getWriter().println(
                "Invalid username or password"
            );
        }
    }
}

```

```

    });
}
}
}

```

A corresponding servlet can read the cookie:

```

@WebServlet("/cookieHome")
public class CookieHomeServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws IOException {

        Cookie[] cookies = request.getCookies();
        String username = null;

        if (cookies != null) {
            for (Cookie cookie : cookies) {
                if ("username".equals(cookie.getName())) {
                    username = cookie.getValue();
                    break;
                }
            }
        }

        if (username != null) {
            response.getWriter().println(

                "<h2>Welcome, " + username + "!</h2>"
            );
        } else {
            response.getWriter().println(
                "Please login first."
            );
        }
    }
}

```

Security note: In a real authentication system, a cookie should generally contain an unpredictable session identifier rather than the username or password. Authentication cookies should use appropriate Secure, HttpOnly, and SameSite protections and be transmitted over HTTPS.

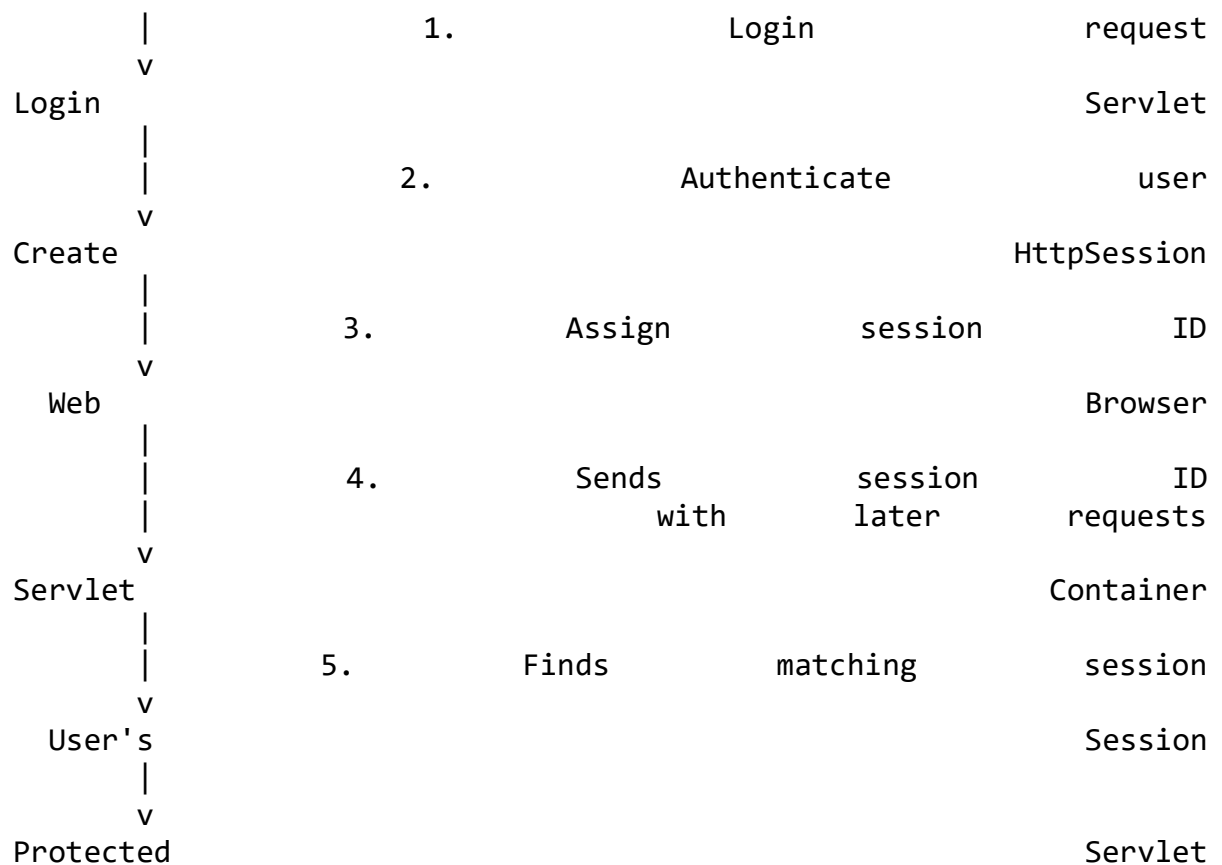
7. How Session Tracking Works

The typical HttpSession process is:

```

User
|

```



Step-by-step process

1. The user submits the login form.
2. The servlet verifies the username and password.
3. If authentication succeeds, the servlet creates an HttpSession.
4. The servlet container assigns a unique **session ID** to that session.
5. Usually, the session ID is sent to the browser using a cookie such as JSESSIONID.
6. The browser sends that session ID with subsequent requests.
7. The servlet container uses the session ID to locate the user's server-side session.
8. The application retrieves the stored user information from the session.
9. When the user logs out, `session.invalidate()` ends the session.

Therefore, although HTTP itself is stateless, session tracking allows the application to maintain the user's state across requests.

8. HttpSession vs Cookies

Feature	HttpSession	Cookies
Storage	Mainly server-side	Client/browser-side

Data size	Can store more application state	Limited size
Security	Sensitive data can remain on server	Data is stored on client
Identification	Usually uses a session ID	Uses cookie name/value
Lifetime	Can be controlled by session timeout	Can be session or persistent
Server memory	Uses server resources	Requires little server storage for cookie data
Browser support	Usually relies on cookies or URL rewriting for session ID	Requires browser cookie support
Ease of use	Easy for storing multiple attributes	Simple for small pieces of client-side data
Suitable for login state	Very suitable	Useful for carrying the session identifier
Example	<code>session.setAttribute("username", "John")</code>	<code>new Cookie("theme", "dark")</code>

9. Advantages of HttpSession

- Easy to use with Java Servlets.
- Data is associated with a particular user.
- Application data can remain on the server.
- Multiple values can be stored as session attributes.
- Supports automatic session management through the servlet container.
- Suitable for authentication and authorization state.

Example:

```
session.setAttribute("username", "John");
session.setAttribute("role", "student");
session.setAttribute("loggedIn", true);
```

10. Advantages and Limitations of Cookies

Advantages

- Simple mechanism for storing small pieces of information.
- Can persist between browser sessions when an expiry time is specified.
- Useful for preferences such as language, theme, or other non-sensitive settings.

- Commonly used to transport a session identifier.

Limitations

- Users can disable cookies.
- Cookies are stored on the client.
- Cookies have size and quantity limitations.
- Cookie values should not be trusted directly because clients can modify them.
- Sensitive information should not be stored directly in ordinary cookies.

11. Which Approach Should Be Used for Login Sessions?

For a login system, **HttpSession** is **generally the preferred application-level mechanism**.

A common design is actually a combination of both:

```

Server:
HttpSession
    |
    | stores username, role, authentication state
    |
    v
Session ID
  
```

```

Browser:
Cookie: JSESSIONID=<session-id>
    |
    | sent with subsequent requests
    |
    v
Server
  
```

The browser stores the session identifier in a cookie, while the actual session information is maintained by the server.

12. Conclusion

Session management is necessary because HTTP is stateless. Java Servlets provide **HttpSession** for maintaining user-specific information across multiple requests. Cookies can be used to store information on the client or, commonly, to carry the session identifier used by the server.

For an authenticated student or user portal, a good design is to use **HttpSession to store authentication state on the server and a secure session cookie such as JSESSIONID to identify the session**. On logout, the session should be invalidated to prevent further access.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

JDBC-Based E-Commerce Product Management System

1. Introduction

An e-commerce web application needs a database to store product information such as:

- Product ID
- Product name
- Price
- Quantity
- Category

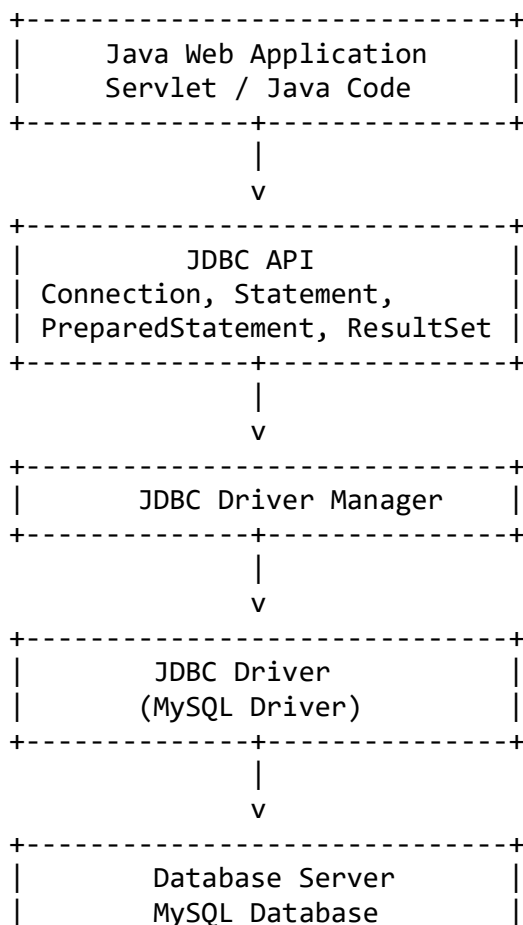
JDBC (Java Database Connectivity) is a Java API used to connect Java applications with relational databases such as MySQL, PostgreSQL, and Oracle.

Using JDBC, a Java web application can:

1. Establish a connection with the database.
2. Execute SQL statements.
3. Insert product information.
4. Retrieve product information.
5. Process query results.
6. Handle database-related exceptions.

2. JDBC Architecture

JDBC follows a layered architecture that allows a Java application to communicate with different databases through JDBC drivers.



+-----+

Main components

1. JDBC API

The JDBC API provides classes and interfaces for database communication.

Important interfaces/classes include:

- DriverManager
- Connection
- Statement
- PreparedStatement
- ResultSet
- SQLException

2. JDBC Driver

A JDBC driver acts as a bridge between the Java application and a particular database.

For example, a MySQL application uses the MySQL JDBC driver.

3. DriverManager

DriverManager manages JDBC drivers and helps establish a connection with the database.

```
Connection con = DriverManager.getConnection(  
    url, username, password  
);
```

4. Connection

The Connection object represents a connection between the Java application and the database.

5. Statement / PreparedStatement

These objects are used to execute SQL statements.

PreparedStatement is preferred for user-supplied data because it helps prevent SQL injection and handles parameter values safely.

6. ResultSet

A `ResultSet` contains the rows returned by a SQL `SELECT` query.

3. Database and Table Design

Assume the e-commerce application uses a MySQL database called `ecommerce`.

```
CREATE DATABASE ecommerce;
```

```
USE ecommerce;
```

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY AUTO_INCREMENT,  
    product_name VARCHAR(100) NOT NULL,  
    price DECIMAL(10,2) NOT NULL,  
    quantity INT NOT NULL,  
    category VARCHAR(50)  
);
```

The table stores product information.

4. Steps Involved in JDBC Database Connectivity

The major steps are:

Step 1: Add JDBC Driver

The appropriate JDBC driver must be available to the Java web application.

For MySQL, the MySQL Connector/J driver is required.

Step 2: Import JDBC Classes

```
import java.sql.*;
```

Step 3: Load/Register the Driver

Modern JDBC drivers are normally automatically discovered when the driver JAR is present, so explicitly calling `Class.forName()` is generally unnecessary.

Older code may use:

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Step 4: Establish Connection

```
Connection con = DriverManager.getConnection(
```

```
        "jdbc:mysql://localhost:3306/ecommerce",  
        "root",  
        "password"  
    );
```

Step 5: Create SQL Statement

Use PreparedStatement for parameterized SQL.

```
PreparedStatement ps =  
    con.prepareStatement(  
        "INSERT INTO products " +  
        "(product_name, price, quantity, category) " +  
        "VALUES (?, ?, ?, ?)"  
    );
```

Step 6: Set Parameters

```
ps.setString(1, "Laptop");  
ps.setDouble(2, 55000.00);  
ps.setInt(3, 10);  
ps.setString(4, "Electronics");
```

Step 7: Execute SQL

For an INSERT, use:

```
ps.executeUpdate();
```

For a SELECT, use:

```
ResultSet rs = ps.executeQuery();
```

Step 8: Process Results

```
while (rs.next()) {  
    System.out.println(rs.getInt("product_id"));  
    System.out.println(rs.getString("product_name"));  
}
```

Step 9: Close Resources

Database resources should always be closed after use.

try-with-resources is recommended because it automatically closes JDBC resources.

5. Java JDBC Application

The following class provides methods to insert and retrieve product information.

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class ProductDAO {

    private static final String URL =
        "jdbc:mysql://localhost:3306/ecommerce";

    private static final String USER = "root";
    private static final String PASSWORD = "password";

    // Insert product
    public void addProduct(String name,
                           double price,
                           int quantity,
                           String category) {

        String sql =
            "INSERT INTO products " +
            "(product_name, price, quantity, category) " +
            "VALUES (?, ?, ?, ?)";

        try (Connection con =
            DriverManager.getConnection(URL, USER, PASSWORD);
            PreparedStatement ps =
                con.prepareStatement(sql)) {

            ps.setString(1, name);
            ps.setDouble(2, price);
            ps.setInt(3, quantity);
            ps.setString(4, category);

            int rows = ps.executeUpdate();

            if (rows > 0) {
                System.out.println("Product inserted successfully.");
            }

        } catch (SQLException e) {
            System.err.println(
                "Database error while inserting product: "
                + e.getMessage()
            );
        }
    }

    // Retrieve products
    public void getProducts() {
```

```

String sql =
    "SELECT product_id, product_name, " +
    "price, quantity, category " +
    "FROM products";

try (Connection con =
    DriverManager.getConnection(URL, USER, PASSWORD);
    PreparedStatement ps =
        con.prepareStatement(sql);
    ResultSet rs =
        ps.executeQuery()) {

    while (rs.next()) {

        int id = rs.getInt("product_id");
        String name = rs.getString("product_name");
        double price = rs.getDouble("price");
        int quantity = rs.getInt("quantity");
        String category = rs.getString("category");

        System.out.println(

            id + " | " + name + " | " +
            price + " | " + quantity + " | " +
            category
        );
    }

} catch (SQLException e) {
    System.err.println(
        "Database error while retrieving products: "
        + e.getMessage()
    );
}
}

```

6. Using the DAO Class

A servlet can use the DAO to insert product information.

```

import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.*;

@WebServlet("/addProduct")
public class ProductServlet extends HttpServlet {

```



```

        protected void doPost(HttpServletRequest request,
        HttpServletResponse response)
            throws ServletException, IOException {

            String name = request.getParameter("name");
            double price =
                Double.parseDouble(request.getParameter("price"));
            int quantity =
                Integer.parseInt(request.getParameter("quantity"));
            String category =
                request.getParameter("category");

            ProductDAO dao = new ProductDAO();

            dao.addProduct(
                name,
                price,
                quantity,
                category
            );

            response.getWriter().println(

                "<h2>Product added successfully</h2>"
            );
        }
    }
}

```

A separate servlet can retrieve and display products.

```

@WebServlet("/products")
public class ProductListServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");

        ProductDAO dao = new ProductDAO();

        // In a production application, the DAO would
        // return product objects rather than print them.
        dao.getProducts();

        response.getWriter().println(
            "<h2>Product list retrieved successfully.</h2>"
        );
    }
}

```

7. Using PreparedStatement

PreparedStatement is preferable to constructing SQL statements by concatenating user input.

Unsafe approach

```
String sql =  
    "SELECT * FROM products WHERE product_name = '"  
    + name + "'";
```

This can create SQL injection vulnerabilities.

Better approach

```
String sql =  
    "SELECT * FROM products WHERE product_name = ?";  
  
PreparedStatement ps =  
    con.prepareStatement(sql);  
  
ps.setString(1, name);
```

The parameter is handled separately from the SQL command.

Therefore, PreparedStatement provides safer and cleaner database access.

8. Exception Handling in JDBC

JDBC operations can generate SQLException.

Common causes include:

- Invalid database URL
- Database server unavailable
- Incorrect username or password
- Invalid SQL query
- Missing table
- Constraint violations
- Connection failures

A basic exception-handling structure is:

```
try {  
    // JDBC operations
```

```
}  
catch (SQLException e) {  
    System.err.println(  
        "Database error: " + e.getMessage()  
    );  
}
```

For debugging, additional information can be obtained with:

```
System.err.println("SQL State: " + e.getSQLState());  
System.err.println("Error Code: " + e.getErrorCode());
```

In a real web application, detailed database errors should normally be logged on the server rather than exposed to users.

9. Why try-with-resources Is Important

JDBC uses resources such as:

- Connection
- PreparedStatement
- ResultSet

These resources should be closed after use.

Instead of manually writing:

```
rs.close();  
ps.close();  
con.close();
```

we can use:

```
try (Connection con = ...;  
    PreparedStatement ps = ...;  
    ResultSet rs = ...) {  
  
    // Database operations  
  
}
```

Java automatically closes the resources when the try block finishes, including when an exception occurs.

This reduces resource leaks and makes the code cleaner.

10. JDBC Data Flow

The complete process can be represented as:

```
User
|
| Product details
v
Web Form
|
| HTTP Request
v
Servlet
|
v
```

```
ProductDAO
|
| JDBC API
v
DriverManager
|
v
JDBC Driver
|
v
MySQL Database
|
| SQL Result
v
ResultSet
|
v
DAO
|
v
Servlet
|
v
HTML Response
|
v
User
```

11. Advantages of JDBC

1. Database independence

The same JDBC API can be used with different relational databases by changing the appropriate driver and connection configuration.

2. Standard API

JDBC provides standard interfaces such as `Connection`, `PreparedStatement`, and `ResultSet`.

3. SQL support

Applications can execute `SELECT`, `INSERT`, `UPDATE`, and `DELETE` statements.

4. Security

`PreparedStatement` helps prevent SQL injection when used correctly.

5. Transaction support

JDBC supports transactions using methods such as:

```
con.setAutoCommit(false);  
con.commit();  
con.rollback();
```

12. Conclusion

JDBC provides the communication layer between a Java web application and a relational database. In an e-commerce application, a servlet can receive product details from a web form and pass them to a DAO, which uses JDBC to insert the data into the database. SQL `SELECT` queries can then retrieve the products using `PreparedStatement` and `ResultSet`.

The basic JDBC process is:

Load/obtain driver → Establish connection → Create `PreparedStatement` → Set parameters → Execute SQL → Process results → Close resources → Handle exceptions.

Using `PreparedStatement`, **try-with-resources**, proper exception handling, and a DAO layer results in a safer, cleaner, and more maintainable JDBC-based web application.